Focus Queue context redesign

FOC-036 · Analysis of task, project, and agent context · 20 October 2026 · Measured against `main` at `a762ed0` and 106 live task contexts

> **Bottom line.** A freshly launched Focus Queue agent reads about **26.6 KB (≈6.7K tokens) of fixed context before it does anything**. FOC-036 reached 44.8 KB. The user's actual request can be as little as **1%** of that. For FOC-035 the request was 286 bytes. The launch prompt repeated it twice inside a 4.9 KB procedural template. A 12.9 KB global `CLAUDE.md` and five must-read files surrounded it.
>
> The fix is structural, not editorial. Give every fact **exactly one owner layer**. Let the host (launcher, CLI, hooks) do what the prompt currently asks the agent to do. Load procedure on demand instead of on every launch. A realistic target is **≈9 KB total and a ≈0.7 KB launch brief**, about a two-thirds reduction. Fix this base layer before the learned `data.md` layer grows, because learned facts on top of a bloated base compound the problem.

1. Where the bytes go today

These are the layers an agent receives when Focus Queue launches it with a prompt. Sizes are measured from files on disk. The launch template was rendered from `Sources/FocusQueueCore/TaskContext.swift:353–400` with real FOC-036 values.

Layer	Source	How it arrives	FOC-035	FOC-036
Global instructions	`~/Documents/CLAUDE.md` (Bias 1.1 KB + Focus Queue contract 6.3 KB + Output Contract 5.6 KB)	Auto-loaded, every session under `~/Documents`	12.9 KB	12.9 KB
Launch prompt: template	`TaskContext.agentPrompt`	First user message	4.9 KB	4.9 KB
Launch prompt: user text	`initialPrompt`	First user message	0.3 KB	6.7 KB
Launch prompt: “Assignment details”	`assignmentDescription` (a copy of the prompt, capped at 2,000 chars)	First user message	0.3 KB	2.0 KB
Placement capsule	`tracking/placements/<launch>.json`	Must-read	1.3 KB	1.3 KB
Project plan	`projects/focus-queue/plan.md`	Must-read	1.3 KB	1.3 KB
Task plan	`tasks/<ID>/plan.md`	Must-read	1.6 KB	2.4 KB
Task state	`tasks/<ID>/context.json`	Must-read (“before acting”)	3.4 KB	12.7 KB
Auto-memory index	`MEMORY.md`	Auto-loaded	0.6 KB	0.6 KB
Total before the first useful action			**≈26.6 KB**	**≈44.8 KB**

FOC-036 is inflated by a 4.8 KB pasted example in the user's own prompt; FOC-035 is the representative case. The repository's `AGENTS.md` (3.1 KB) is not counted: Claude Code loads `CLAUDE.md`, and the repository has none. Codex loads `AGENTS.md`, so Codex agents carry it in addition.

Share of the FOC-035 launch that is the user's request



Green: the user's 286-byte request (1.1%). Orange: the same request repeated as “Assignment details”. Everything else is procedure, identity, or machine state.

Corpus findings across 106 task contexts

- **45 subtasks** store `assignmentDescription` equal to (or a prefix of) `initialPrompt`, so the launch prompt shows the request twice.
- Long prompts appear **2–6 times** inside one `context.json`: `initialPrompt`, `assignmentDescription`, `summary`, and archive snapshots. The largest `context.json` is 305 KB.
- **48/106** tasks carry the generic success criterion *“Confirm the requested behavior and record verification results.”* It costs bytes in every prompt and tells the agent nothing.
- **9/106** plans carry the filler *“Why: This task advances the recorded project goal.”*. Generated BLUFs paste the raw prompt as “Current focus”: FOC-035's BLUF contains the 286-byte request with its typos.
- The rendered template repeats the support-directory path **14 times**, the agent UUID **11 times**, and the ticket ID **15 times**. The launcher already exports all three as environment variables.

2. Anatomy of the current launch prompt

Each sentence of `agentPrompt` classified by what it does and whether a prompt is the right carrier for it. KEEP stays in the brief. MOVE becomes host behavior or on-demand text. CUT duplicates another owner.

#	Current text (abridged)	Verdict	Reason and new home
1	The user's prompt	KEEP	The only irreplaceable content. Send it once, verbatim, and first.
2	“Work on FOC-035: <title>. Your subtask is <subtask title> (<UUID>).”	KEEP, shorter	Task ID and title orient the agent. The subtask title usually equals the task title; print it only when it differs.
3	“Project: … Primary repository: …”	KEEP	One short line.
4	“Read project guidance at <path> and the placement capsule at $FOCUS_QUEUE_AGENT_CONTEXT before acting.”	MOVE	The capsule already lists every path. Use one “Read first” line, or inject the documents with a hook.
5	“Assignment details: <prompt again>”	CUT	It duplicates line 1 for 45 subtasks. Emit it only when an orchestrator assigned text that differs from the prompt.
6	“Read the central plan … and context.json before acting. If the plan is missing, create … (BLUF, What/Why/How, …). Keep it concise; re-read … Link supporting canvases.”	CUT	The host always creates the plan, so “if missing” never fires. The section list repeats the Output Contract. `context.json` is machine state; use `focus-queue-session context` for a digest.
7	“Keep decisions, detailed progress … in <agent-log path>. Append with <full command> … do not inject the whole log …”	MOVE	The plan-versus-log rule already lives in the global contract, the plan template, and `practices.md`. The brief needs only the `log` verb.

8	“Briefly state your assignment, provider, workspace, branch, and intended outputs. Reuse this worktree on resume. Create an isolated worktree if none is assigned …”	CUT	The launcher already created the worktree; the conditional is dead text. The status recital costs a turn and restates what the brief just said.
9	“Workspace: … Assigned branch: …”	KEEP	Identity. One line.
10	“Success criteria: …”	KEEP if specific	Omit when it equals the generic default (48/106 tasks).
11	“Register this pane: FOCUS_QUEUE_PROVIDER=… FOCUS_QUEUE_TASK_ID=… FOCUS_QUEUE_SUBTASK_ID=… FOCUS_QUEUE_SUPPORT_DIR=… attach …”	MOVE	The host can register the pane. Claude Code accepts `--session-id <uuid>`, so the launcher can choose the UUID and record it before launch. A SessionStart hook covers the other cases.
12–15	progress / attention / result / complete / idle / blocked, each with the full env-prefixed command and both IDs	KEEP, compressed	Keep the verbs and drop the arguments. The CLI should default the task and agent IDs from `FOCUS_QUEUE_TASK_ID` and `FOCUS_QUEUE_SUBTASK_ID`, which are already exported. That turns ~1.8 KB into ~250 bytes.
16	Map-scope instructions (`focus-queue-map scope …`), 2 sentences	MOVE	Rarely relevant at launch. Move them to `focus-queue-session rules` or infer scope from tool events in a hook.
17	review-push paragraph explaining what the orchestrator scans	KEEP 1 line	Keep “Before any push: `review-push`; push only when authorized.” The scanner's internals belong in its own output.
18	“Update the plan within scope … Ask before changing scope … workspace is immutable … use `workspace` only to …”	CUT	This duplicates the global contract, `AGENTS.md`, and the project plan. The `workspace` command is for the host and orchestrator, not a feature agent.
19	“Complete only your assigned subtask. Preserve other agents' changes. Never force-remove …”	CUT	This appears in 4 other places (see §3).

The FOC-035 example in the request has one line printed twice (“Complete only your assigned subtask … Never force-remove a”) and a truncated review-push sentence. The template emits that text once (`TaskContext.swift:399`), so those defects look like terminal-wrap copy artifacts, not generator bugs. Not verified against the raw tmux buffer.

3. The same rule, many owners

Each row is one rule. Each column is a file an agent receives or must read. Every extra • costs bytes on every launch, and each copy can drift. The stale WezTerm wording in `AGENTS.md` already shows that drift.

Rule	Global CLAUDE.md	Launch prompt	Repo AGENTS.md	Project plan.md	practices.md	Task plan.md	Queue-chat prompt*
Register / attach before work	••	•	•				
Workspace is immutable	•	••	•	••	•		
Install only from canonical checkout	•		•	••	•		

Integration: ff-only, no force-push	•		•		•		
Plan vs. agent-log split	•	••		•	•	••	•
Output-contract section list	•	•				•	•
Sanity-test before commit	••		•		•		
Push only when authorized	•	•	•				
Preserve other agents' work	•	••		•	•		
Labels / tmux presentation	•		•				
Never hand-edit state JSON	•						•

* The orchestrator/queue-chat instructions at `Sources/FocusQueueCore/Integrations.swift:2540` restate the plan/log/output-contract rules in a further 1.6 KB paragraph. •• means the rule appears twice in the same file.

Prior work that already points in this direction

- **Single-sourced contract** (`kimi/agent-contract-refresh`, commit `ce627e2`): generates the global payloads from `Config/agent_contract/` and trims them to ~3.5–4 KB. Unmerged and 56 commits behind `main`. Revive its generator; this proposal shrinks the payload further.
- **Proactive context routing lab** (FOC-014, `experiment/proactive-context-routing`): defines destination rules (prompt / user / project / task / deferred) and a ledger. Unmerged and 56 commits behind. Reuse its admission rules for the learned layer (§6).
- **Task** `data.md` (FOC-010, commit `6fe8068`): creates `tasks/<ID>/data.md` and adds it to the capsule and prompts. It is 1 commit ahead of `main`. It currently adds a sentence to *both* prompt templates, the pattern this proposal wants to stop.

4. File-by-file: what exists, what should be there, what to clean

File (owner layer)	Currently contains	Should contain	Clean up
`~/Documents/CLAUDE.md` (user, global)	Bias for action; the full 6.3 KB Focus Queue backend contract; the 5.6 KB Agent Output Contract with a worked example.	Only cross-project user preferences: bias for action, the output contract, and a ≤3-line Focus Queue trigger.	**MOVE** the Focus Queue contract out. It loads in every session under `~/Documents`, including non-Focus-Queue work, and half of it (shell entry, placement, orchestrator) is host behavior. Optional: compress the worked example (~1.5 KB).

`AGENTS.md` (repo)	UI-automation opt-in; WezTerm label rules; delivery and integration rules; hard-coded `/Users/hira/.local/bin` path.	Repository-wide rules for *any* agent, Focus Queue or not: UI-automation opt-in, how to build and test, and delivery rules for this repo.	**CUT** the attach and WezTerm sections (now tmux-managed and host-owned). **ADD** a repo `CLAUDE.md` containing `@AGENTS.md`, so Claude agents load one shared owner.
`projects/<p>/plan.md` + `practices.md` (project)	Two heavily overlapping lists of workspace, install, and ID rules. "Project commands" lists only the repository path.	One `project.md` of ≤1.5 KB: purpose (1 line), **build/test/install commands**, project-specific practices, and avoid-list. Learned facts go in project `data.md`.	**CUT** rules already owned by `AGENTS.md` or the global layer. **ADD** the missing commands: `swift build`, `swift test`, the Python tests, and the install script.
`prd.md`, `architecture.md`, `evidence.md` (project)	Reasonable reference documents.	Unchanged, but read on demand only and never listed as must-read.	None. Link them from `project.md`.
`tasks/<ID>/plan.md` (task)	BLUF with the raw prompt as "Current focus"; a boilerplate Why; two generic decision-log entries; generic QA; generic next step.	≤800 bytes: Goal as what/why/how in 1–2 sentences each, specific success criteria, current focus and next step, and links. Other sections appear only once they have real content.	**CUT** the template filler. **MOVE** summary generation to a one-shot model rewrite (`claude -p`), as FOC-035's "What" line already shows.
`tasks/<ID>/context.json` (host state)	Prompt copies (×2–6), tmux PIDs, heartbeats, workspace metadata, and archive pointers.	Machine state only. Store the prompt once, keep `assignmentDescription` only when it differs, and derive `summary` rather than truncating the prompt.	**CUT** it from every "read before acting" instruction. Agents use `focus-queue-session context`, which can print a 10-line digest.
`agent-log.md` (task)	Append-only history (median 139 B, max 159 KB).	Unchanged: write-mostly, read-on-demand.	Add rotation or summarization above ~20 KB so "read relevant entries" stays cheap.
`data.md` (task; FOC-010)	A one-line header.	Learned, durable task facts: inputs, test data, gotchas. Bounded at ~2 KB with dated entries.	Reference it through the capsule only, not through new prompt sentences.

Placement capsule (agent)	IDs, paths, and tmux identity (1.3 KB). Already well designed.	The canonical pointer map for the agent.	Add `taskDataPath` and `projectDataPath`, and make it the only path source the brief references.

5. Target design: one owner per fact

Layer	Owns	Carrier	Budget	Learned slot
User (global)	How the user wants agents to behave and report: bias for action, the output contract, and personal preferences.	`~/Documents/CLAUDE.md`, `~/.codex/AGENTS.md` (generated from one source)	≤5 KB	User `data.md` (or Claude memory) for cross-project preferences
Focus Queue runtime	The lifecycle verbs, push gating, and workspace immutability: rules that apply only inside a managed pane.	`--append-system-prompt` or a SessionStart hook; `focus-queue-session rules` for detail	≤1 KB	None (product behavior, versioned in the repo)
Repository	Build, test, UI-automation, and delivery rules for this codebase.	`AGENTS.md + CLAUDE.md → @AGENTS.md`	≤3 KB	None (edited by commits)
Project	Purpose, commands, project-specific practices; links to PRD, architecture, evidence.	`project.md` (merged plan + practices)	≤1.5 KB	Project `data.md`: UI style, conventions, recurring gotchas
Task	Goal, success criteria, current focus, next step.	`plan.md`	≤0.8 KB	Task `data.md`; history goes in `agent-log.md`
Agent	Identity (IDs, workspace, branch) and the user's request.	Launch brief + capsule	≤0.7 KB + prompt	None (ephemeral)

Admission test: where does a sentence belong?

1. **Can the host do it instead?** Then it is code, not context: registration, worktree creation, default IDs, plan creation, label management.
2. **Is it true for every session the user runs?** Then it belongs in the user layer.
3. **Is it true only inside a Focus Queue pane?** Then it belongs in the runtime layer, loaded only when `FOCUS_QUEUE_TASK_ID` is set.
4. **Is it about this codebase for any agent?** Then `AGENTS.md`. If it is about this project's goals and conventions, `project.md`. If it is about this objective, the task plan.
5. **Was it learned rather than designed?** Then the matching `data.md`, never a template.
6. **Is it already said by a higher layer?** Then delete it.

Proposed launch brief (FOC-035, ≈0.7 KB of procedure)

```
The Qchat currently does not properly render markdown. Please ensure it can proerply render md. also,
for some reason, i cannot select and copy several lines within the chat, due to some kind of
formatting issue. pls resolve these and ensure changes are properly committed and installed

[Focus Queue] FOC-035 “Fix Qchat markdown rendering and selection” · project Focus Queue
Agent DD6DF4D3 (claude) · workspace .worktrees/foc-035/agents/dd6df4d3-… · branch focus-queue/foc-035-
dd6df4d3-…
Read first: task plan and task data (paths in $FOCUS_QUEUE_AGENT_CONTEXT).
Report with focus-queue-session <verb> (IDs come from the environment):
  progress working|idle|blocked "…" · attention approval|question "…" · result "…" · complete "…" ·
log "…"
Before any push run focus-queue-session review-push; push only when the user authorizes it.
Full rules on demand: focus-queue-session rules
```

Compare the current ~5.5 KB. The user's text stays first and unmodified, so titles, summaries, and transcript recall derived from the first message stop absorbing boilerplate. With Claude, the bracketed block can move into `--append-system-prompt` so it is not a user turn at all. Codex and Kimi keep it in the prompt.

6. Mechanisms that make the shrink safe

Deleting text is only safe if the behavior it asked for still happens. Each mechanism below replaces prompt text with something deterministic.

Mechanism	Replaces	Implementation notes
Env-defaulted CLI arguments	14 support-dir prefixes, 11 UUIDs, and 15 ticket IDs in the template (~2 KB).	`progress`, `attention`, `result`, `complete`, `log`, and `review-push` fall back to `FOCUS_QUEUE_TASK_ID` / `FOCUS_QUEUE_SUBTASK_ID` when arguments are omitted. The helper already defaults the support directory (`Integrations.swift:2540`), and the launcher exports it (`AgentLauncher.swift:150`).
Host-side registration	The “Register this pane …” line and two global-contract bullets about session IDs.	For Claude, pass `--session-id <uuid>` at launch and record it with the tmux pane before the process starts. Keep `attach` as the fallback for Codex and Kimi, and for manual shells.
`focus-queue-session rules` (new)	Map scope, workspace refresh, the review-push explanation, the integration and cleanup rules, and the Focus Queue half of the global contract.	Generated from the same single source as the contract payloads (revive `Config/agent_contract/generate.py`). Agents read it when a rule becomes relevant.
Runtime-layer delivery	The 6.3 KB Focus Queue contract in global `CLAUDE.md`.	Global keeps: “If `$FOCUS_QUEUE_TASK_ID` is set, you are a managed Focus Queue agent; follow the launch brief and `focus-queue-session rules`.” The brief carries the rest.
Host-written plans	“If the plan is missing, create … BLUF, What/Why/How …”.	The host already creates `plan.md`. Generate its Goal with one `claude -p` rewrite of the prompt (subscription CLI, not API keys), as the user's model-call preference requires.

Byte-budget tests	Future drift.	A Swift test asserts that `agentPrompt` minus the user text is ≤900 bytes and contains no path more than once. A Python test asserts the generated global payload is ≤5 KB.

7. Learned-data layer, after the base is clean

- **Three scopes, one shape.** `user/data.md`, `projects/<p>/data.md`, and `tasks/<ID>/data.md`. Each holds dated, sourced bullets under a byte cap (user 2 KB, project 3 KB, task 2 KB).
- **Write path.** A `focus-queue-session remember "<fact>" [--scope task|project|user]` verb, as FOC-010's summary already proposes. Without `--scope`, a small model applies FOC-014's admission rules. Deterministic checks enforce identity and the cap.
- **Read path.** The capsule lists the paths. The brief says “Read first: task plan and task data”, and project `data.md` is read with `project.md`. User data lives in the global layer, so it is already loaded.
- **Hygiene.** When a scope exceeds its cap, a compaction pass merges duplicates and retires superseded facts into `agent-log.md`. Learned facts never get copied into templates.

8. Migration plan

Phase	Change	Verification	Saves per launch
1	Rewrite `agentPrompt` / `agentBootstrap` as the brief in §5. Drop the duplicate assignment, the generic success criteria, and the `context.json` must-read. Add env defaults to the CLI.	Update `AgentLauncherTests.swift:29`, `SubtaskFocusTests.swift:447`, and `TaskContextTests`. Add the byte-budget test. Launch one real agent on a scratch task and confirm attach, progress, and complete land in the queue.	≈4.5 KB + context.json
2	Move the Focus Queue contract out of global `CLAUDE.md` into `rules` and the runtime layer. Revive the single-source generator. Add repo `CLAUDE.md → @AGENTS.md` and drop the stale WezTerm and attach text from `AGENTS.md`.	Generator drift test. A non-Focus-Queue Claude session no longer contains the contract. A managed agent still registers and reports.	≈6 KB (every session)
3	Host-side registration via `--session-id` for Claude. Merge `plan.md` + `practices.md` into `project.md` with commands. Slim the task-plan template and add model-written goals.	Registration rate on 5 real launches. Project hydration preserves authored bytes (existing invariant).	≈2 KB
4	Learned layer: three-scope `data.md`, the `remember` verb, caps, and compaction. Build it on FOC-010 and FOC-014.	Replay FOC-014 fixtures. Spot-check placements on live prompts in shadow mode.	Bounded growth

Risks

- **Agents skip lifecycle calls once instructions get shorter.** Mitigate with host-side registration, a Stop hook that reports `idle`, and a dashboard measure of the attach/complete rate before and after Phase 1.

- **Provider differences.** Kimi takes the prompt non-interactively via `--prompt`, and Codex has no `--append-system-prompt`. The brief must work as plain prompt text for both.
- **Old branches.** The contract-refresh and routing branches are 56 commits behind. Port the ideas; do not merge them blindly.

9. Decisions for you

1. **Approve Phase 1 as the next task?** It is self-contained, inside `FocusQueueCore`, test-covered, and removes most of the visible repetition.
2. **Where should the Focus Queue runtime rules live for Claude:** `--append-system-prompt` (invisible, deterministic) or the visible brief (portable across providers)? Recommendation: the visible brief for all providers first, then the system prompt for Claude.
3. **Keep the Output Contract's worked example in global** `CLAUDE.md`**?** It is the largest optional block (~1.5 KB), but it is your style guide, so this is your call.
4. **Merge** `plan.md` **and** `practices.md` **into one** `project.md`**?** This changes the project-document set that hydration creates.

Method: sizes from `wc -c` on live files. The launch template was rendered with a script that substitutes FOC-036 values into `TaskContext.swift`. Corpus statistics come from a scan of every `tasks/*/context.json` and `plan.md` under the support directory. Token estimates use 4 bytes ≈ 1 token. Session-start overhead from Claude Code itself (system prompt, tool schemas) is excluded because Focus Queue does not control it.